

Visualização: Raytracing

Ana Carolina Hora

Fevereiro de 2026

1 Novas Primitivas: Cubo e Cilindro

Ambas as classes foram implementadas em `src/shapes.py`, modeladas em coordenadas naturais com origem no centro de massa e eixos alinhados às características do objeto.

Cubo

A interseção raio-cubo é resolvida pelo *slab method*: para cada par de faces paralelas ($\pm x$, $\pm y$, $\pm z$), calcula-se o intervalo $[t_1, t_2]$ de interseção com os planos correspondentes. O intervalo final de interseção é a intersecção de todos os intervalos:

$$t_{\min} = \max(t_{1x}, t_{1y}, t_{1z}), \quad t_{\max} = \min(t_{2x}, t_{2y}, t_{2z}).$$

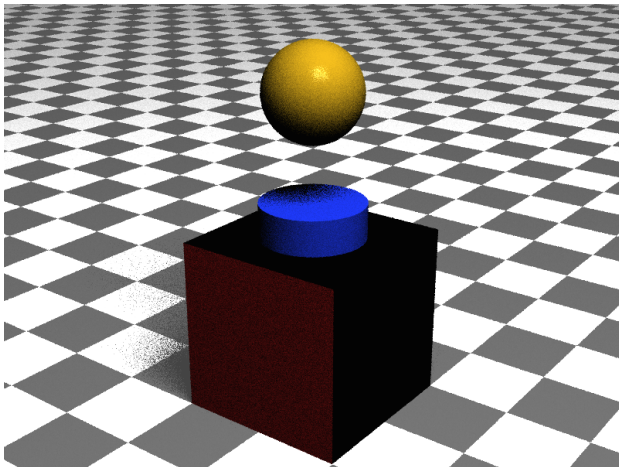
Há interseção quando $t_{\min} \leq t_{\max}$ e $t_{\min} > \varepsilon$. A normal é determinada pelo eixo que produziu o maior t_1 (face de entrada). A semi-extensão é configurável via parâmetro `half_size`.

Cilindro

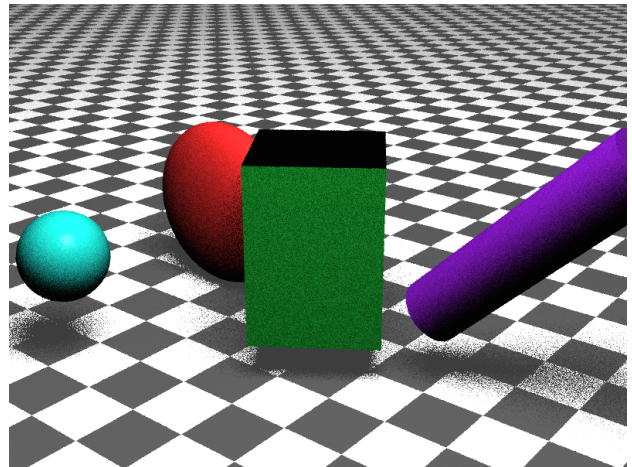
O cilindro (eixo Z , raio r , semi-altura h) é dividido em dois componentes:

1. **Corpo lateral** – equação quadrática $x^2 + y^2 = r^2$, restrita a $|z| \leq h$. A normal é radial: $\mathbf{n} = (x/r, y/r, 0)$.
2. **Tampas** – interseção com os planos $z = \pm h$, verificando $x^2 + y^2 \leq r^2$. Normais axiais $\pm \hat{z}$.

Todos os candidatos são computados e seleciona-se o de menor t positivo.



(a) Cubo (vermelho), Cilindro (azul) e esfera dourada sobre chão xadrez.



(b) ObjectTransform: parabolóide (esfera escalada), cubo rotacionado 35° e cilindro inclinado 45°.

Figure 1: Tarefas 1 e 2.

2 ObjectTransform

A classe `ObjectTransform` encapsula qualquer `Shape` com uma transformação afim:

$$\mathbf{p}_{\text{mundo}} = M \mathbf{p}_{\text{obj}} + \mathbf{T},$$

onde M é uma matriz 3×3 arbitrária e $\mathbf{T}$ é uma translação. O método `hit()` opera em três etapas:

(A) **Levar o raio ao espaço do objeto.**

$$\mathbf{o}_{\text{obj}} = M^{-1}(\mathbf{o}_{\text{mundo}} - \mathbf{T}), \quad \mathbf{d}_{\text{obj}} = \frac{M^{-1}\mathbf{d}_{\text{mundo}}}{\|M^{-1}\mathbf{d}_{\text{mundo}}\|}.$$

A direção é transformada *sem normalizar antes da divisão* para que a escala de t seja preservada corretamente.

(B) **Trazer o ponto de interseção ao espaço do mundo.**

$$\mathbf{p}_{\text{mundo}} = M \mathbf{p}_{\text{obj}} + \mathbf{T}, \quad t_{\text{mundo}} = \frac{t_{\text{obj}}}{\|M^{-1}\mathbf{d}_{\text{mundo}}\|}.$$

(C) **Por que $M^{-\top}$ para a normal?** A normal $\mathbf{n}$ satisfaz $\mathbf{n} \cdot \mathbf{t} = 0$ para qualquer tangente $\mathbf{t}$ na superfície. Após a transformação M , os tangentes viram $M\mathbf{t}$. Queremos $\mathbf{n}'$ tal que $\mathbf{n}' \cdot (M\mathbf{t}) = 0$. Escrevendo $\mathbf{n}' = A\mathbf{n}$:

$$(A\mathbf{n})^\top (M\mathbf{t}) = \mathbf{n}^\top A^\top M \mathbf{t} = 0 \quad \forall \mathbf{t} \implies A^\top M = I \implies A = (M^{-1})^\top = M^{-\top}.$$

Logo $\mathbf{n}_{\text{mundo}} = (M^{-\top} \mathbf{n}_{\text{obj}}) / \|\dots\|$.

A cena de teste (Figura 1b) contém um parabolóide (esfera escalada anisotropicamente com $s_x = 1.5$, $s_y = 0.6$, $s_z = 1.5$), um cubo rotacionado 35° em Z com escala em Y , e um cilindro inclinado 45° em torno de Y e esticado em Z . A iluminação Phong com sombras confirma a correção das normais.

3 Superfícies Implícitas

A classe `ImplicitSurface` renderiza $f(x, y, z) = 0$ com o seguinte algoritmo:

1. **Teste raio–AABB.** Se o raio não acerta a caixa delimitadora, retorna imediatamente (sem calcular f).
2. **Amostragem uniforme.** Divide o segmento interior em $n_{\text{steps}} = 80$ amostras e avalia f em cada uma.
3. **Bisseção.** Em cada intervalo com mudança de sinal de f , aplica-se bisseção com 48 iterações (precisão $\approx 10^{-14}$), permitindo múltiplas raízes por raio.
4. **Normal.** Computada pelo gradiente numérico de f por diferenças centradas com $\varepsilon = 10^{-4}$:

$$\nabla f \approx \left(\frac{f(x+\varepsilon)-f(x-\varepsilon)}{2\varepsilon}, \frac{f(y+\varepsilon)-f(y-\varepsilon)}{2\varepsilon}, \frac{f(z+\varepsilon)-f(z-\varepsilon)}{2\varepsilon} \right).$$

As funções implementadas foram:

$$f_{\text{Mitchell}}(x, y, z) = 4(x^4 + (y^2 + z^2)^2 + 17x^2(y^2 + z^2)) - 20(x^2 + y^2 + z^2) + 17$$

$$f_{\text{Coração}}(x, y, z) = \left(x^2 + \frac{9}{4}y^2 + z^2 - 1 \right)^3 - x^2z^3 - \frac{9}{80}y^2z^3$$

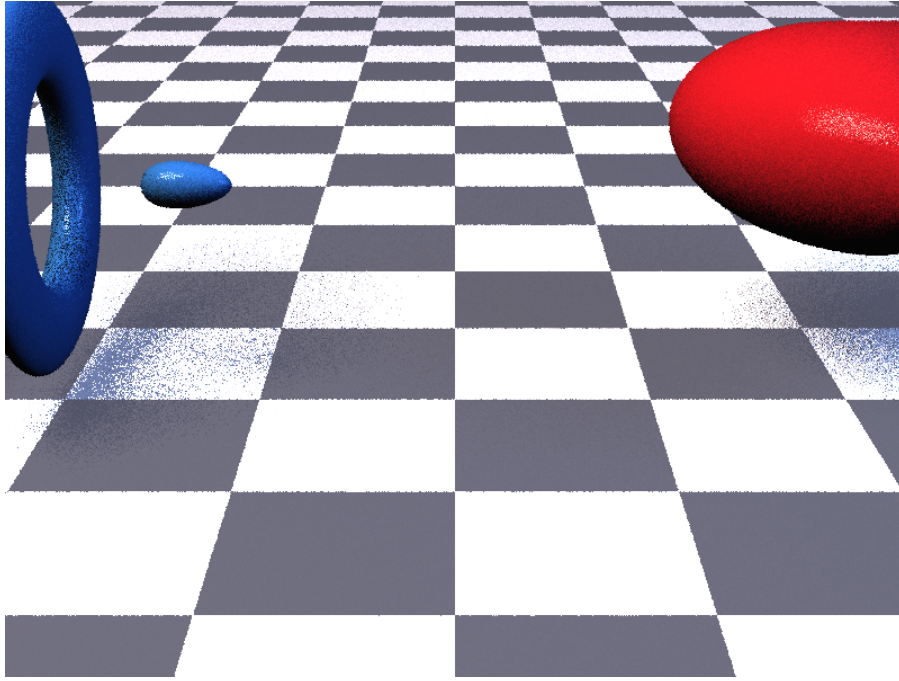


Figure 2: Superfície de Mitchell (azul, esquerda) e superfície Coração (vermelha, direita), posicionadas via `ObjectTransform`.

Observação importante: as funções f foram definidas no nível do módulo (não como *closures*) para compatibilidade com `multiprocessing` via `pickle`.

4 Material Espelho

O `MirrorMaterial` implementa reflexão pura e recursiva. A direção refletida é:

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n}) \mathbf{n},$$

garantindo que $\mathbf{n}$ aponte contra o raio incidente. A recursão é limitada por `max_depth`.

Com dois espelhos paralelos opostos e a câmera entre eles, o número de reflexões visíveis é aproximadamente `max_depth - 1` (cada ida-e-volta consome dois níveis de recursão).

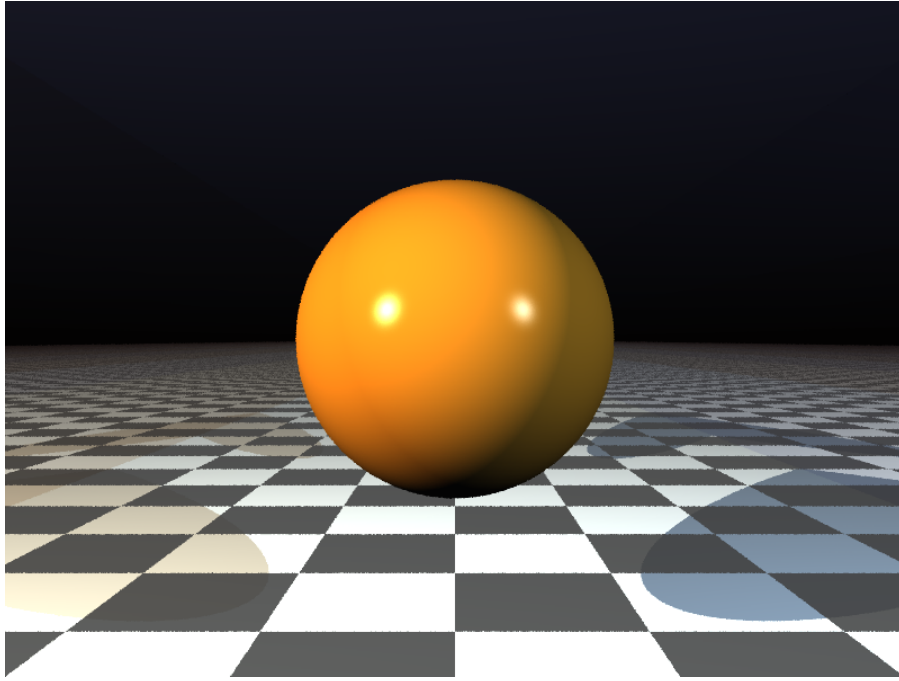


Figure 3: Câmera entre dois espelhos paralelos ($\text{max_depth}=12$). Reflexões da esfera laranja visíveis no chão e nas paredes virtuais.

5 Profundidade de Campo

A `ThinLensCamera` estende a câmera pinhole com dois parâmetros: `lens_radius` (raio da abertura R) e `focal_dist` (distância do plano focal d_f).

Para cada raio primário:

1. Calcula-se o **ponto focal**: interseção do raio primário com o plano a distância d_f da câmera.
2. Amostra-se uma **posição aleatória** no disco da lente (método da rejeição): $\mathbf{l} = \mathbf{e} + R(u \hat{\mathbf{u}} + v \hat{\mathbf{v}})$, com $u^2 + v^2 \leq 1$.
3. O raio final parte de $\mathbf{l}$ em direção ao ponto focal.

Objetos no plano focal convergem (aparecem nítidos); os demais são borrados proporcionalmente à distância ao plano focal.

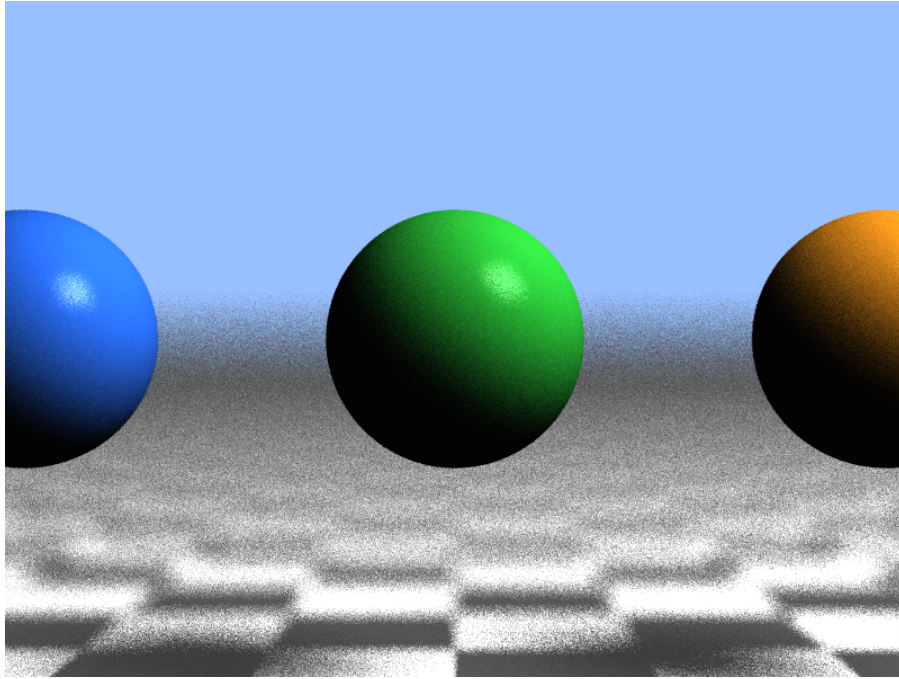


Figure 4: Fileira de 5 esferas coloridas a distâncias distintas. Foco na esfera verde (centro). $\text{lens_radius}=0.35$, $\text{focal_dist}=10$, $n=16$ amostras/pixel.

6 Dificuldades Encontradas

- **Pickle de closures no multiprocessing.** Funções definidas dentro de outras funções (*closures*) não podem ser serializadas pelo módulo `multiprocessing`. A solução foi mover as funções implícitas f para o nível do módulo.
- **Escala de t no ObjectTransform.** Transformar a direção do raio *sem normalizar* antes da interseção é essencial: a norma de $M^{-1}\mathbf{d}$ define o fator de escala entre t_{obj} e t_{mundo} . Normalizar prematuramente produzia pontos de interseção incorretos.
- **Desempenho das superfícies implícitas.** Com $n_{\text{steps}} = 80$ e 48 iterações de bisseção por intervalo, a Tarefa 3 levou ≈ 140 s para 800×600 com 2 amostras e 4 workers. Uma melhoria possível seria ray marching adaptativo ou redução do número de iterações de bisseção.
- **Ruído no DOF.** Com poucas amostras ($n < 16$) o efeito de bokeh fica muito granulado, pois cada pixel amostra posições distintas na lente. Foram necessárias 16 amostras/pixel para resultado visualmente aceitável.